



A Top-N Recommender for Biological Papers



Ebrahim Alhaddad, Manish Sampath, Trevor Sullivan, Arman Soleimani

1. Problem Statement + Predictive Task

Motivation:

The complexity of biological subfields creates barriers to entry

Predictive Task:

Recommend the top-N papers a biologist should read

Evaluation + Baselines:

Hit Rate @ K

Global Average, User Bias, Item Bias



2. Data Collection + Pre-Processing

Dataset:
Queried OpenAlex for journal articles with the following params:

- 1. Published after Jan 01, 2024
- 2. Springer Nature (publisher ID: P4310319965)
- 3. Biology (concepts.id:C86803240)

Returned 38203 papers which gave us:

Ref. Authors	Ref. Papers	Interactions	Density
219525	1538115	15012345	4.446e-05

```
import requests
import time
from collections import defaultdict

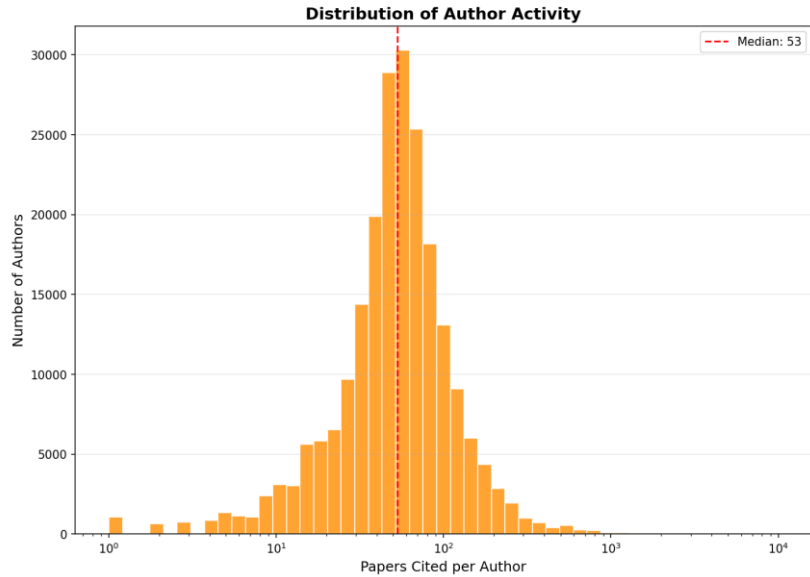
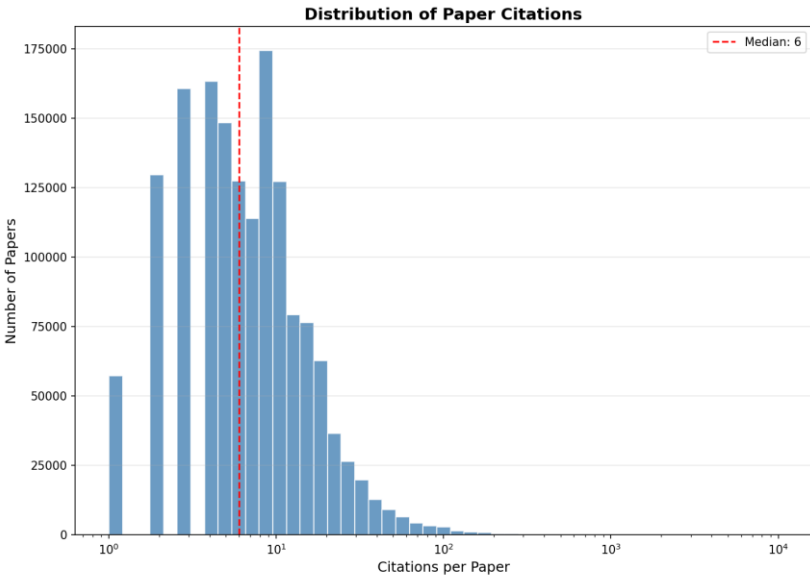
BASE_URL = "https://api.openalex.org/works"

# Springer Nature + biology + 2024+ + must have references
# FILTER = ",".join([
#     "primary_location.source.publisher_lineage:P4310319965",
#     "concepts.id:C86803240",
#     "type:journal-article",
#     "from_publication_date:2024-01-01", # Changed this line
#     "has_references:true",
# ])

FILTER = ",".join([
    "primary_location.source.publisher_lineage:P4310319965",
    "concepts.id:C86803240",
    "from_publication_date:2024-01-01", # Changed this line
    "has_references:true",
])

params = {
    "filter": FILTER,
    "per-page": 200,
    "cursor": "*",
    "select": "id,authorships,referenced_works,publication_year",
}
```

Basic User-Item Stats:



2. Data Collection + Pre-Processing

```
62 # -----  
63 # FETCH FUNCTION  
64 # -----  
65 async def fetch_one(session, wid, semaphore):  
66     url = f"https://api.openalex.org/authors/{wid}"  
67     params = {"select": FIELDS}  
68  
69     async with semaphore:  
70         for attempt in range(5):  
71             try:  
72                 async with session.get(url, params=params, timeout=TIMEOUT) as resp:  
73  
74                     if resp.status == 429:  
75                         await asyncio.sleep(float(resp.headers.get("Retry-After", "1")))  
76                         continue  
77  
78                     if resp.status >= 400:  
79                         err_text = await resp.text()  
80                         return wid, None, f"{resp.status}: {err_text}"  
81  
82                     return wid, await resp.json(), None  
83  
84             except Exception as e:  
85                 await asyncio.sleep(1.5 * attempt)  
86  
87         return wid, None, "Max retries exceeded"
```

2. Data Collection + Pre-Processing

```
90 # -----
91 # ASYNC MAIN
92 # -----
93 async def main_fetch(work_ids):
94
95     semaphore = asyncio.Semaphore(CONCURRENT_REQUESTS)
96     timeout = aiohttp.ClientTimeout(sock_connect=TIMEOUT, sock_read=TIMEOUT)
97
98     async with aiohttp.ClientSession(timeout=timeout) as session:
99
100         tasks = [fetch_one(session, wid, semaphore) for wid in work_ids]
101
102         for task in tqdm_asyncio.as_completed(tasks, total=len(tasks)):
103             wid, data, err = await task
104
105             if data is not None:
106                 # append result
107                 async with aiofiles.open(RESULTS_FILE, "a") as f:
108                     await f.write(json.dumps(data) + "\n")
109             else:
110                 # append error
111                 async with aiofiles.open(ERROR_FILE, "a") as f:
112                     await f.write(json.dumps({"id": wid, "error": err}) + "\n")
113
114             # update seen ids
115             seen_ids.add(wid)
116             async with aiofiles.open(SEEN_FILE, "a") as f:
117                 await f.write(wid + "\n")
118
```

2. Data Collection + Pre-Processing

Preprocessing Steps:

1. Required that a paper need 10 or more citations
2. Required an author to have cited at least 20 papers

Raw Data

Authors	Papers	Interactions	Density
219525	1538115	15012345	4.446e-05



① Used for EDA

Filtered Data

Authors	Papers	Interactions	Density
190728	466769	9692022	0.0001088



Model Data

② Used for Models

Authors	Papers	Interactions	Density
10000	346165	1149697	0.0003321

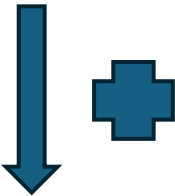
2. Data Collection + Pre-Processing

- Features:
- Scraped author metadata from OpenAlex
 - Pre-processed metadata to fit our model
 - Generated popularity scores for works

Model Data

2 Used for Models

Authors	Papers	Interactions	Density
10000	346165	1149697	0.0003321



Author

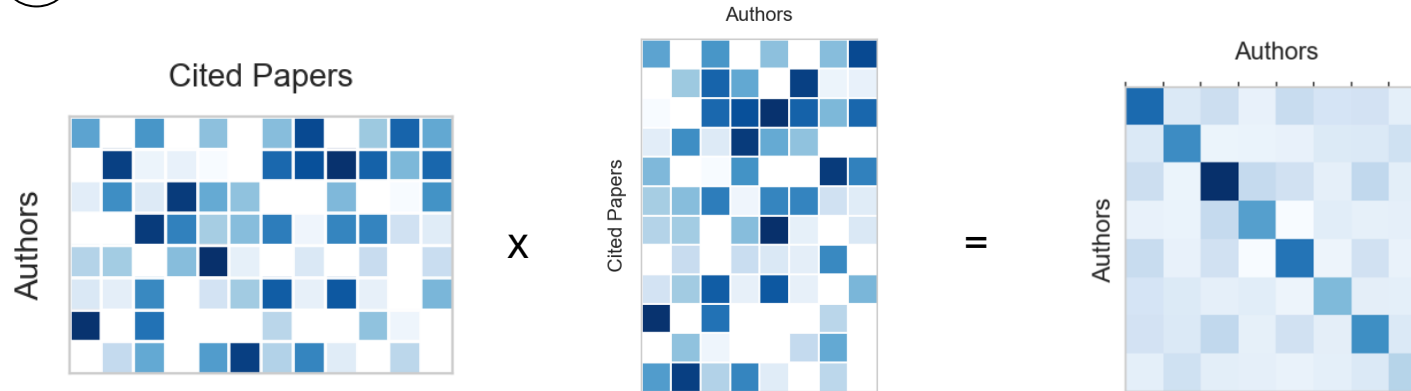
Feature	Measure
works_count	Level of activity
cited_by_count	popularity
last_active_year	Time decay
concepts	Content similarity

Work

Feature	Measure
local_pop	Popularity within model data

2. Exploratory Analysis

1 Author to Author Similarity



```
def build_csr(filtered_pairs):
    pairs = np.asarray(list(filtered_pairs))
    unique_authors, user_idx = np.unique(pairs[:, 0], return_inverse=True)
    unique_papers, item_idx = np.unique(pairs[:, 1], return_inverse=True)
    data = np.ones(len(pairs), dtype=np.float32)
    mat = sparse.coo_matrix(
        (data, (user_idx, item_idx)), shape=(len(unique_authors), len(unique_papers))
    )
    author_to_row = {aid: i for i, aid in enumerate(unique_authors)}
    paper_to_col = {pid: i for i, pid in enumerate(unique_papers)}
    return mat.tocsr(), author_to_row, paper_to_col

mat, author_rows, paper_columns = build_csr(filtered_pairs_set)
AA = mat @ mat.T # Author-Author co-citation
PP = mat.T @ mat # Paper-Paper co-citation
print(f"Author matrix: {AA.shape}, Paper matrix: {PP.shape}")
```

2 SVD

```
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import normalize
from umap import UMAP

def svd_umap(M, n_svd=64, n_samples=20000, random_state=42):
    M = M.tocsr()
    n = M.shape[0]

    if n > n_samples:
        idx = np.random.choice(n, n_samples, replace=False)
        M_sub = M[idx]
    else:
        idx = np.arange(n)
        M_sub = M

    svd = TruncatedSVD(n_components=n_svd, random_state=random_state)
    X_svd = svd.fit_transform(M_sub)
    X_svd = normalize(X_svd)

    umap = UMAP(
        n_components=2,
        metric="cosine",
        random_state=random_state,
        n_neighbors=15,
        min_dist=0.1,
    )
    X_umap = umap.fit_transform(X_svd)

    return idx, X_svd, X_umap

author_idx, author_svd, author_umap = svd_umap(AA, n_svd=64, n_samples=20000)
paper_idx, paper_svd, paper_umap = svd_umap(PP, n_svd=64, n_samples=20000)
```

3 KMeans Clustering

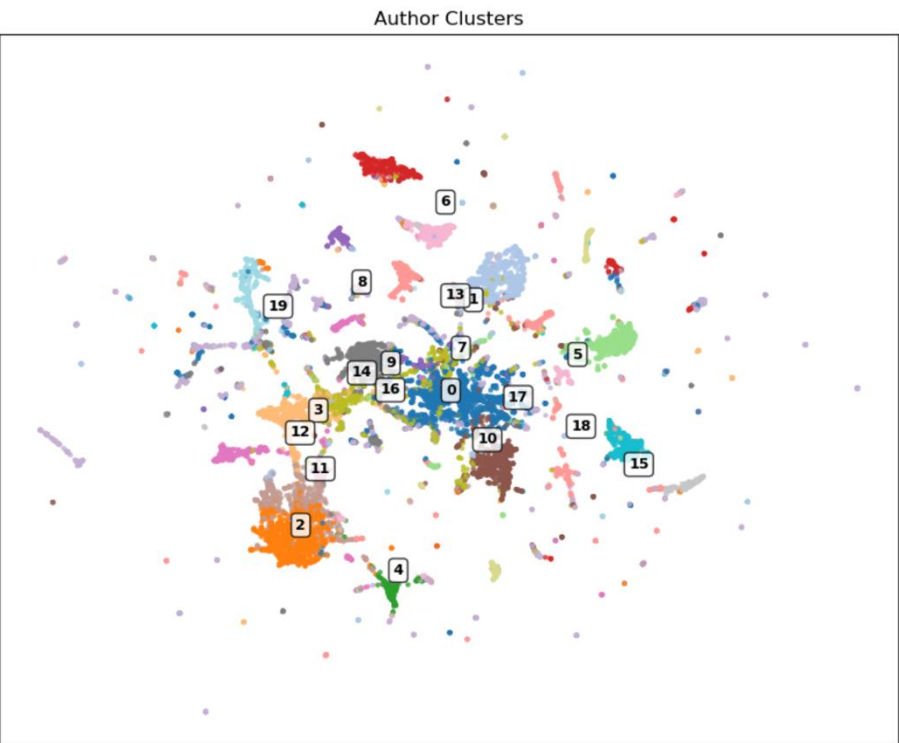
```
from sklearn.cluster import KMeans

n_clusters = 20

author_kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
author_cluster_labels = author_kmeans.fit_predict(author_svd)

paper_kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
paper_cluster_labels = paper_kmeans.fit_predict(paper_svd)
```


2. Exploratory Analysis



```
import requests
from tqdm import tqdm

row_to_author = {row: aid for aid, row in author_rows.items()}
embedded_author_ids = [row_to_author[r] for r in author_idx]

def chunks(lst, n):
    for i in range(0, len(lst), n):
        yield lst[i : i + n]

def fetch_author_fields(author_ids, batch_size=50):
    id_to_field = {}
    for batch in tqdm(list(chunks(author_ids, batch_size))):
        id_filter = "|".join(batch)
        params = {"filter": f"ids.openalex:{id_filter}", "per-page": len(batch)}
        r = requests.get("https://api.openalex.org/authors", params=params)
        r.raise_for_status()
        for a in r.json()["results"]:
            topics = a.get("topics", [])
            if topics:
                top = topics[0]
                domain = top.get("domain", {}).get("display_name", "")
                field = top.get("field", {}).get("display_name", "Unknown")
                subfield = top.get("subfield", {}).get("display_name", "Unknown")
                field_name = (
                    subfield
                    if domain == "Life Sciences" or "Biology" in field
                    else field
                )
            else:
                field_name = "Unknown"
            id_to_field[a["id"]] = field_name
    return id_to_field

author_field = fetch_author_fields(embedded_author_ids)
```

20 rows x 17 columns						
cluster	field_1	field_2	field_3	field_4	field_5	
0	Medicine	Molecular Biology	Engineering	Immunology	Materials Science	
1	Environmental Science	Medicine	Molecular Biology	Plant Science	Engineering	
2	Medicine	Molecular Biology	Cancer Research	Engineering	Immunology	
3	Medicine	Molecular Biology	Physics and Astronomy	Engineering	Environmental Science	
4	Medicine	Molecular Biology	Immunology	Genetics	Cancer Research	
5	Medicine	Molecular Biology	Immunology	Environmental Science	Engineering	
6	Medicine	Molecular Biology	Immunology	Cancer Research	Engineering	
7	Medicine	Molecular Biology	Cancer Research	Engineering	Materials Science	
8	Medicine	Molecular Biology	Engineering	Computer Science	Genetics	
9	Medicine	Genetics	Molecular Biology	Cellular and Molecular Neuroscience	Engineering	
10	Medicine	Genetics	Molecular Biology	Psychology	Cognitive Neuroscience	
11	Medicine	Molecular Biology	Engineering	Cancer Research	Materials Science	
12	Medicine	Engineering	Environmental Science	Nursing	Molecular Biology	
13	Medicine	Molecular Biology	Genetics	Engineering	Computer Science	
14	Engineering	Environmental Science	Plant Science	Business, Management and Accounting	Medicine	
15	Medicine	Plant Science	Molecular Biology	Environmental Science	Engineering	
16	Medicine	Molecular Biology	Cancer Research	Engineering	Immunology	
17	Plant Science	Environmental Science	Engineering	Medicine	Materials Science	
18	Medicine	Molecular Biology	Engineering	Materials Science	Cancer Research	
19	Environmental Science	Plant Science	Engineering	Materials Science	Medicine	

bioinformatics

neuroscience

biophysics

3. Modeling



Model	Optimization
Unbiased SVD	$\hat{r}_{ui} = \mathbf{q}_i^\top \mathbf{p}_u$
NMF	$\hat{r}_{ui} = \mathbf{q}_i^\top \mathbf{p}_u$ where $\mathbf{p}_u, \mathbf{q}_i \geq 0$
Baselines (Popularity / Random)	$\hat{r}_{ui} \sim U(0, 1)$ $\hat{r}_{ui} = \frac{\text{count}(i)}{\max_j \text{count}(j)}$
Factorization Machines	$\hat{r}_{ui} = \sigma \left(\left(\sum_{j \in f_u} e_j \right)^\top \left(\sum_{k \in f_i} e_k \right) \right)$

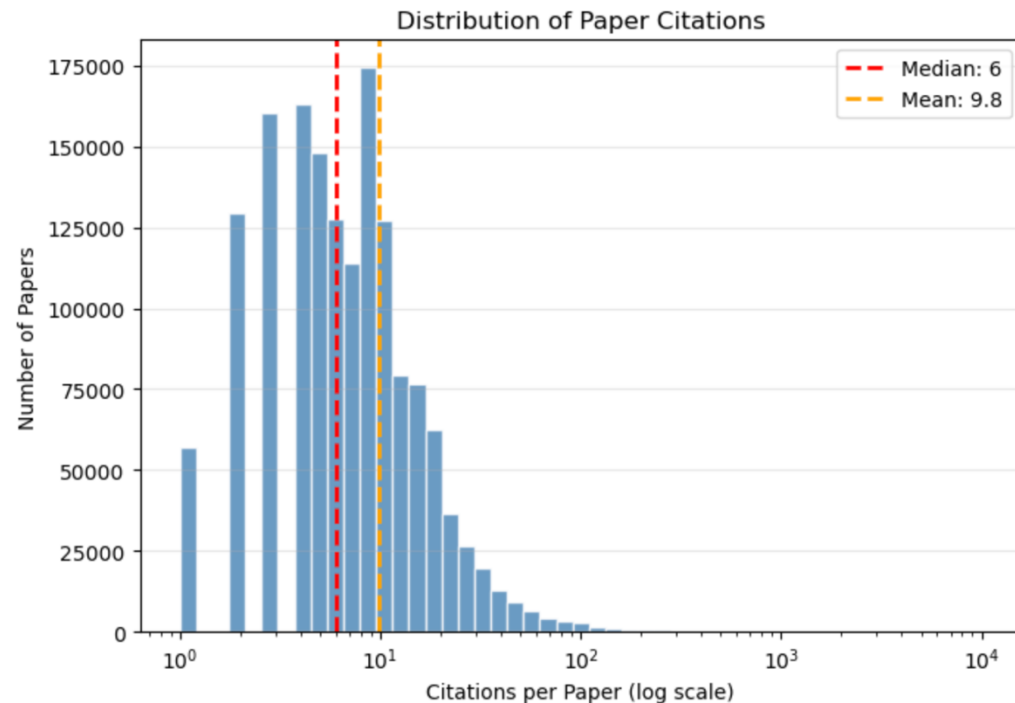
3. Modeling: Random & Popularity Baselines

Random: $\hat{r}_{ui} \sim U(0, 1)$

- Assigns uniform random scores
- Lower bound on performance

Popularity: $\hat{r}_{ui} = \frac{\text{count}(i)}{\max_j \text{count}(j)}$

- Ranks items by global citation frequency
- Non-personalized baseline



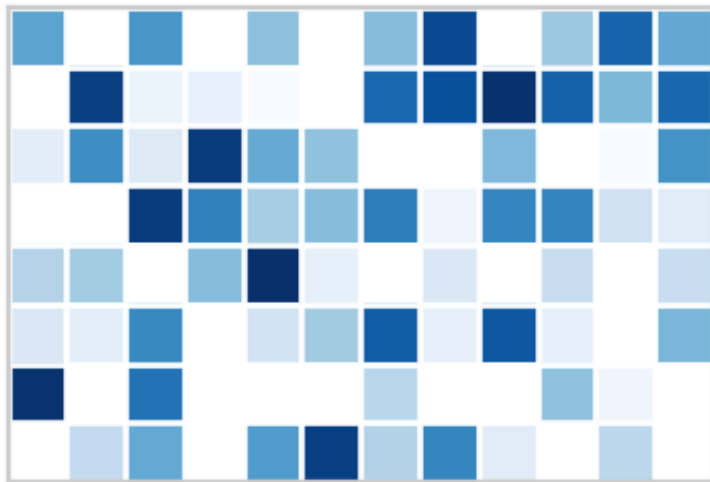
```
1 def run_baselines(data, k_values=[10, 15, 20, 25, 30], num_neg=50000, max_users=10000):
2     train_user_items = data["train_user_items"]
3     test_user_items = data["test_user_items"]
4     n_items = data["n_items"]
5     item_counts = data["item_counts"]
6
7     rng = np.random.default_rng(69)
8     all_items = np.arange(n_items)
9
10    random_hits = {k: 0 for k in k_values}
11    pop_hits = {k: 0 for k in k_values}
12    total = 0
13
14    for idx, (u, pos_items) in enumerate(test_user_items.items()):
15        if idx >= max_users:
16            break
17        if not pos_items:
18            continue
19
20        pos_array = np.array(list(pos_items))
21        train_items = train_user_items.get(u, set())
22        neg_pool = np.setdiff1d(all_items, np.array(list(pos_items | train_items)))
23
24        sample_size = min(num_neg, len(neg_pool))
25        if sample_size == 0:
26            continue
27
28        sampled_neg = rng.choice(neg_pool, size=sample_size, replace=False)
29        candidates = np.concatenate([pos_array, sampled_neg])
30
31        random_scores = rng.random(len(candidates))
32        pop_scores = -item_counts[candidates]
33
34        for k in k_values:
35            k_eff = min(k, len(candidates))
36            top_random = set()
37            candidates[np.argsort(-random_scores)[-k_eff:]]
38
39            if any(p in top_random for p in pos_array):
40                random_hits[k] += 1
41            top_pop = set(candidates[np.argsort(-pop_scores)[-k_eff:]]
42            if any(p in top_pop for p in pos_array):
43                pop_hits[k] += 1
44        total += 1
45
46    print(f"{'K':<5} {'Random':<12} {'Popularity':<12}")
47    for k in k_values:
48        print(f"{'K':<5} {random_hits[k] / total:<12.4f} {pop_hits[k] / total:<12.4f}")
49
50
51 run_baselines(data)
```

3. Modeling: Matrix Factorization

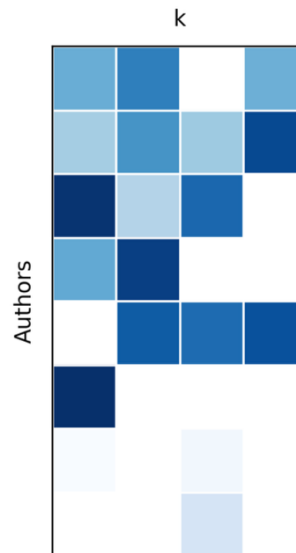
$$\hat{r}_{ui} = \mathbf{q}_i^\top \mathbf{p}_u$$

$$\sum_{r_{ui} \in R_{\text{train}}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|\mathbf{q}_i\|^2 + \|\mathbf{p}_u\|^2).$$

Cited Papers



=



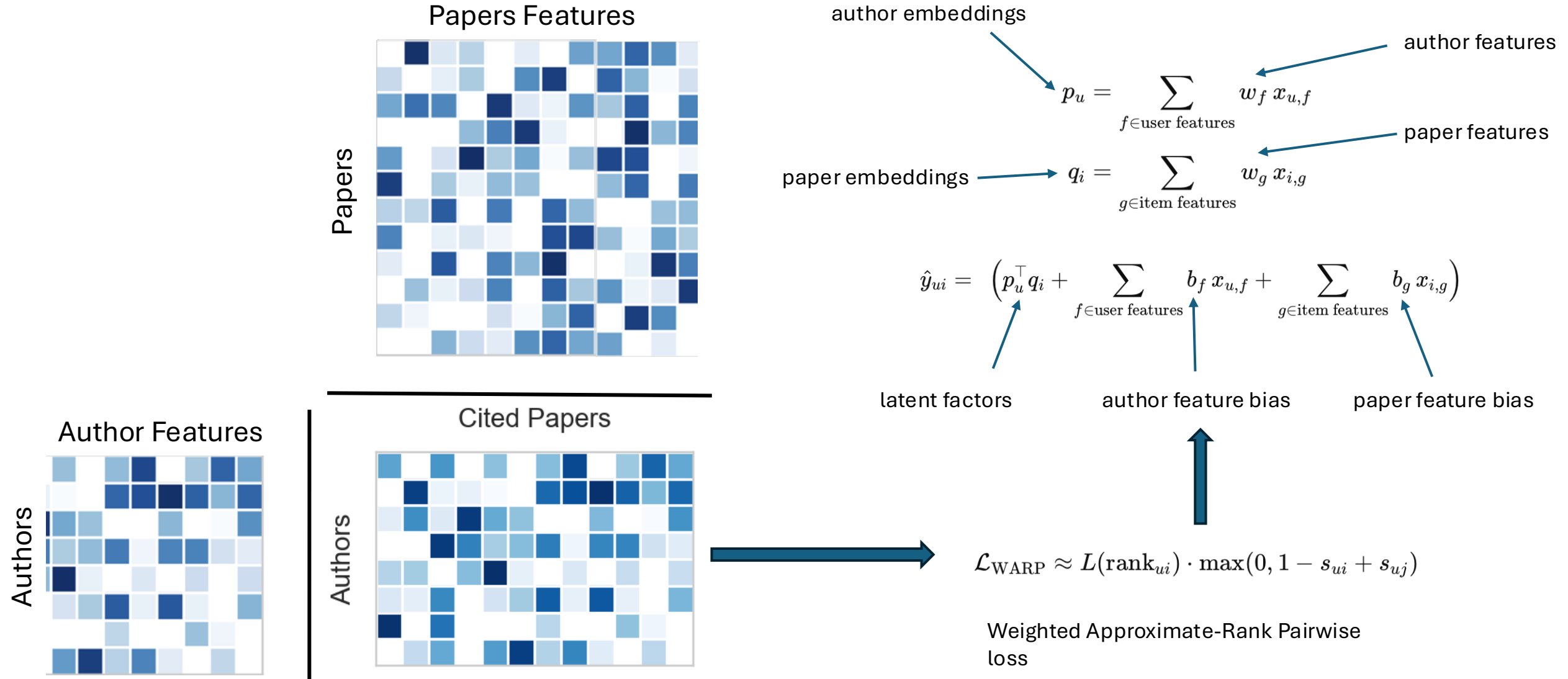
Papers



```
df = generate_df(sampled_pairs)
dataset = load_dataset(df)
trainset, testset = surprise.model_selection.train_test_split(
    dataset, test_size=0.2, random_state=69
)

for name, algo in [
    (
        "SVD",
        surprise.SVD(
            n_factors=64,
            n_epochs=40,
            lr_all=0.005,
            reg_all=0.02,
            random_state=69,
            biased=False,
        ),
    ),
    ("NMF", surprise.NMF(n_factors=64, n_epochs=40, random_state=69)),
]:
    print(f"\n{name}:")
    algo.fit(trainset)
    for k in [10, 15, 20, 25, 30]:
        print(f"  HR@{k}: {hr_at_k(algo, trainset, testset, k):.4f}")
```

3. Modeling: Factorization Machines (LightFM)



3. Modeling: LightFM

- All features are converted to one-hot encoding
- Identity matrix is used to represent users and differentiate feature rows
- Input size increases drastically with added features
 - Number of latent factor components must increase during training
 - Without proper component number, the model will regress significantly

author one-hot encoding feat 1 one-hot encoding feat 2 one-hot encoding

Author 1	1	0	0	0	0	0	1	0	0	1
Author 2	0	1	0	0	0	0	1	0	0	1
Author 3	0	0	1	0	1	0	0	1	0	0
Author 4	0	0	0	1	0	1	0	0	1	0

```
cited_rows = user_meta.index.values
cited_cols = user_meta["cited_bucket"].values
cited_data = np.ones(n_users)

cited_matrix = csr_matrix(
    (cited_data, (cited_rows, cited_cols)),
    shape=(n_users, 6),
    dtype=np.float32
)

year_rows = user_meta.index.values
year_cols = user_meta["year_bucket"].values
year_data = np.ones(n_users)

year_matrix = csr_matrix(
    (year_data, (year_rows, year_cols)),
    shape=(n_users, 5),
    dtype=np.float32
)

user_identity = identity(n_users, dtype=np.float32, format="csr")

user_features = hstack(
    # [user_identity, works_matrix, year_matrix],
    [user_identity, year_matrix],
    format="csr"
)
```

4. Evaluation

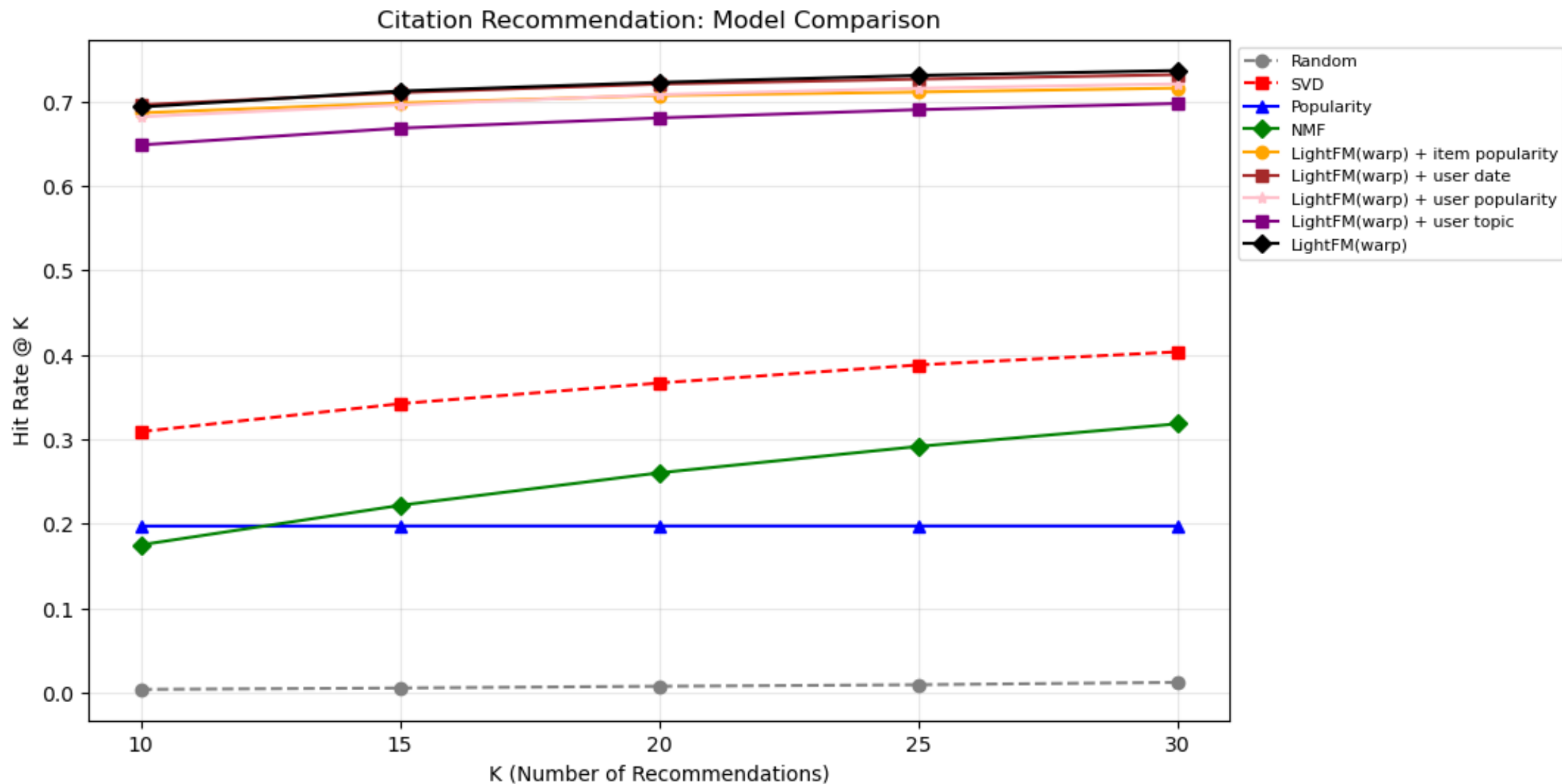
Hits@K Hits@K denotes the ratio of the test triples that have been ranked among the top k triples, i.e.,

$$\text{Hits@k} = \frac{|\{t \in \mathcal{K}_{test} \mid \text{rank}(t) \leq k\}|}{|\mathcal{K}_{test}|} \quad (42)$$

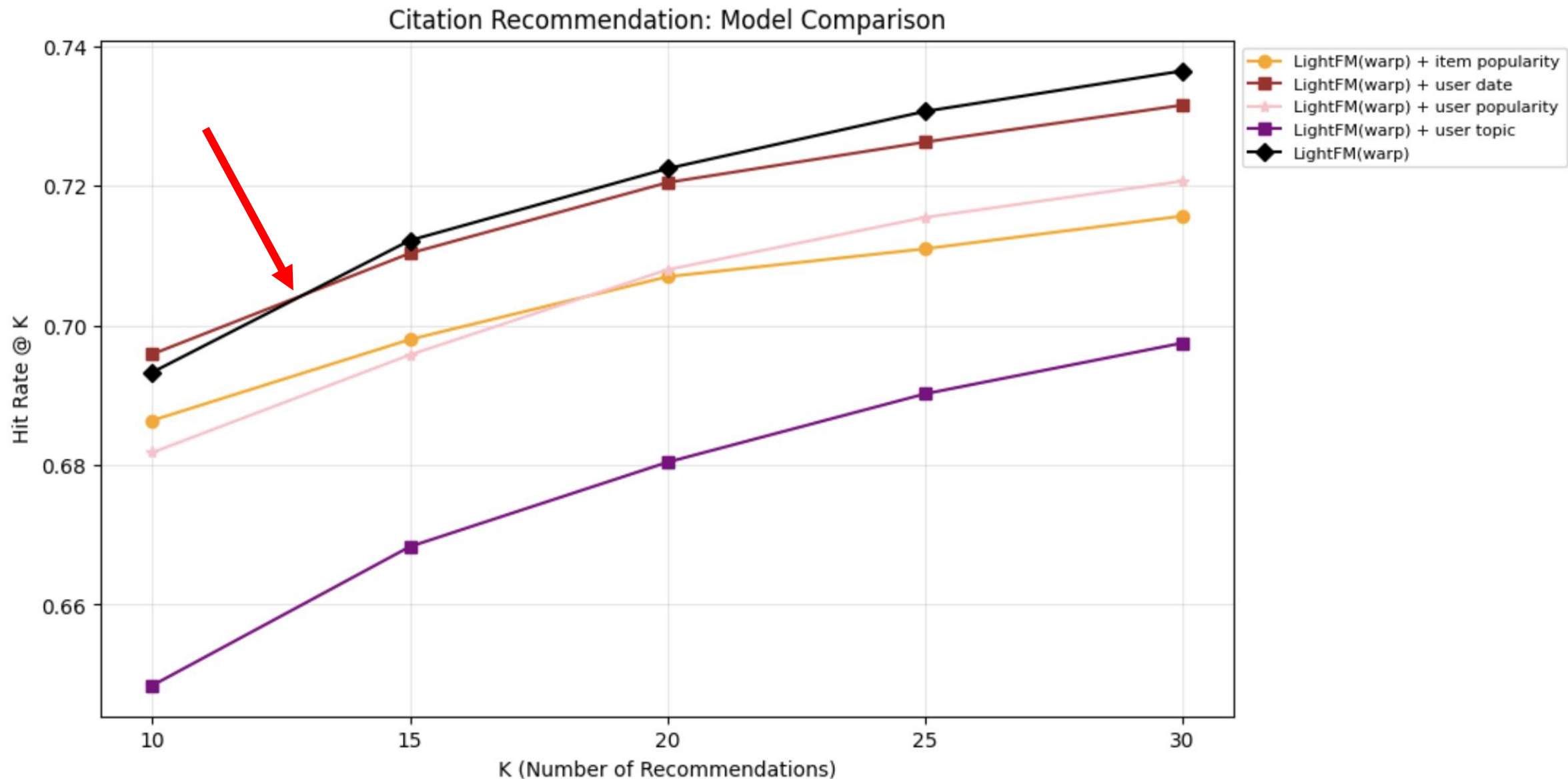
Larger values indicate better performance.

Hit Rate @ K					
K	10	15	20	25	30
Random	0.41%	0.57%	0.78%	0.96%	1.24%
Popularity	19.73%	19.74%	19.74%	19.74%	19.74%
SVD Unbiased	30.91%	34.22%	36.67%	38.81%	40.36%
NMF	17.51%	22.19%	26.04%	29.17%	31.84%
FM	69.3%	71.2%	72.2%	73%	73.7%
FM + last_active_year	69.6%	71%	72%	72.6%	73.2%

4. Evaluation



4. Evaluation



5. Discussion

- **1. Why LightFM Wins**
- WARP loss designed for ranking, not rating prediction
- Handles implicit feedback directly (no need to treat as regression)
- Hybrid approach leverages both interactions AND features
- **2. Why SVD > NMF**
- SVD captures both positive and negative latent patterns
- NMF's non-negativity constraint limits expressiveness
- Biases in SVD help model user/item popularity effects
- **3. Why Popularity is Strong**
- Power-law citation distribution (few papers get most citations)
- Non-personalized but exploits data skew effectively
- **4. Limitations**
- Cold start: new authors/papers have no interaction history
- Scalability: 346K papers required sampling to 10K authors
- Implicit only: doesn't capture citation strength/context

5. Discussion

Possible Improvements:

- Experiment with different combinations of user/item features
- Feature Engineering
 - Normalize or split popularity scores by concepts (Areas of interest)
 - Analyze trends related to time-decay
 - Conduct TF-IDF and latent factors on text
 - Average all authors' papers
 - Embedding per paper
 - Compare performance of our generated features (model data only) compared to those pulled from OpenAlex
- Use NeuFM or DeepFM to improve Collaborative Filtering performance
- Construct a more nuanced pipeline
 - Content-based filtering with metadata
 - Dense embedding similarity (MMR) with metadata filtering
 - TF-IDF based (BM25) with metadata filtering
 - Generate a large pool of candidates (~1000)
 - Use collaborative filtering to rank candidates & find top-k